

AI Assignment 2 Report

Variational Bayesian Last Layers for Handwritten Digit Classification

CISC3024 Pattern Recognition | October 2026 | AI-assisted study and reproduction

Executive summary

This report documents an AI-assisted study of Variational Bayesian Last Layers (VBLL), a Bayesian classification method published at ICLR 2024 [1]. The AI assistant helped screen the literature, verify the primary paper and author repository, generate and run a small implementation, analyze the outputs, and write this report. The implementation places an independent diagonal-Gaussian posterior over the class weights in a neural network's final layer while keeping the learned feature extractor deterministic.

The evaluation uses the scikit-learn digits dataset and a matched deterministic multilayer perceptron (MLP) baseline. Beyond the initial seed-42 split, the experiment was repeated over ten predeclared stratified splits and initialization seeds (11, 23, 42, 57, 99, 123, 2024, 2025, 3024, 4096). Across those ten runs, VBLL's mean accuracy was $96.96 \pm 0.72\%$, compared with $96.64 \pm 0.84\%$ for the MLP. The average paired accuracy difference was +0.31 percentage points, with a sample standard deviation of 0.63 points.

The ten-run result is mixed rather than a blanket win: VBLL had higher mean NLL (0.1108 ± 0.0283 versus 0.1062 ± 0.0302), higher mean Brier score (0.0503 ± 0.0101 versus 0.0490 ± 0.0101), and higher mean ECE (0.0256 ± 0.0049 versus 0.0187 ± 0.0056). This is useful evidence about the assignment implementation, but the small benchmark and overlapping repeated test splits do not establish a general ranking of the methods.

Test metric	MLP mean $\pm$ SD	VBLL mean $\pm$ SD	Preferred
Accuracy	$96.64 \pm 0.84\%$	$96.96 \pm 0.72\%$	higher
NLL (nats)	0.1062 ± 0.0302	0.1108 ± 0.0283	lower
Brier score	0.0490 ± 0.0101	0.0503 ± 0.0101	lower
ECE (15 bins)	0.0187 ± 0.0056	0.0256 ± 0.0049	lower
Error AUROC	0.9587 ± 0.0224	0.9512 ± 0.0185	higher

Ten stratified holdout runs. SD is the sample standard deviation across seeds; accuracy is shown as a percentage.

Choosing a method through AI-assisted search

The assignment asks for a recent computer-vision or pattern-recognition classification method grounded in probability or Bayesian theory, and explicitly expects AI to handle searching, programming, and report writing. The AI first extracted those constraints from the supplied assignment PDF. Its recorded search queries were “Variational Bayesian Last Layers ICLR 2024 classification,” “VBLL official implementation variational Bayesian last layer code,” and “Variational Bayesian Last Layers ELBO formula classification.” It screened methods for a recent peer-reviewed primary source, an explicit probabilistic formulation, and publicly available author code, and shortlisted VBLL alongside other practical Bayesian last-layer approaches.

VBLL was selected because its ICLR 2024 paper presents a probabilistic distribution over the final-layer parameters and a deterministic variational classification objective, while the authors provide a public implementation [1, 2]. That combination makes the method recent, explicitly Bayesian, classifiable with a standard neural feature network, and feasible to reproduce on an offline toy-sized image dataset. The choice was a feasibility and traceability decision; no head-to-head literature-search experiment was performed at this stage.

Screening question	Evidence used	Decision for this study
Does it meet the topic?	Probabilistic posterior over final-layer weights for classification [1]	Yes: directly connected to Bayesian pattern recognition
Is it recent and citable?	ICLR 2024 proceedings record identifies the authors, venue, and paper [1]	Yes: use the primary proceedings page as the main citation
Can implementation be checked?	Public author-maintained package and documentation [2]	Yes: use it to verify provenance; implement a scoped reproduction
Can it run locally?	Digits is a small built-in image dataset with ten classes [3]	Yes: no external data download is needed

Representative prompts used

Literature search

Prompt given to the AI

Find a recent classification method for computer vision or pattern recognition that is explicitly based on probability or Bayesian theory. Prefer a 2023-or-later peer-reviewed primary source and an author-maintained implementation. Explain why it fits this assignment and what can be reproduced on a small offline dataset.

Paper and method check

Prompt given to the AI

Check the candidate against the ICLR proceedings page and the paper's classification method. State which component is Bayesian, what is deterministic, and what is a practical small-data reproduction rather than a replication of the authors' full experiments.

Experiment plan

Prompt given to the AI

Design a reproducible digits experiment with a deterministic baseline, stratified train/validation/test splits, checkpoint selection on validation data only, probability-quality metrics, and a clear statement that the test data are not used for tuning.

The AI checked the proceedings record for the paper identity and abstract, then checked the author-maintained repository for implementation provenance. The assignment prompt and additional prompts for code generation, result interpretation, and report drafting are preserved in AI_PROMPTS.md in the linked repository. Prompt text is included as an audit trail; the cited sources, rather than the AI's unsupported recollection, are the authority for the algorithm.

A probabilistic view of the output layer

Let x be a flattened 8×8 image and let $\phi(x; \theta)$ be the 32-dimensional ReLU feature vector from a trained MLP, augmented with a constant 1 to represent the output bias. The final linear layer maps the 33-dimensional augmented feature into $K = 10$ class logits. A standard MLP estimates one fixed matrix W . VBLL instead approximates uncertainty in that matrix with a variational distribution $q(W)$; it does not make the feature extractor Bayesian in this reproduction.

$$\begin{aligned} q(W) &= \text{product over classes } k \text{ of } \text{Normal}(w_k \mid \mu_k, \text{diag}(\sigma_k^2)) \\ p(W) &= \text{product over classes } k \text{ of } \text{Normal}(w_k \mid \theta, I) \end{aligned}$$

Each class row has its own mean vector and diagonal variance vector. The prior is an independent zero-mean Gaussian with variance 1.0. The implementation parameterizes posterior variances through trainable log variances, initialized to $\log(0.1)$, and clamps log variance to $[-12, 4]$ before exponentiation for numerical stability. The independent diagonal form is a tractable approximation: it cannot represent correlations among weights.

For one-hot label y_n and augmented feature ϕ_n , the classification theorem uses the mean logits and their variances to obtain a deterministic lower bound. With zero output-noise variance, the bound averaged over N training examples is:

$$L = (1/N) \sum_n [y_n^T M \phi_n - \log \sum_k \exp(m_k^T \phi_n + 0.5 \phi_n^T S_k \phi_n)] - \text{KL}(q(W) \parallel p(W)) / N$$

Here m_k and $S_k = \text{diag}(\sigma_k^2)$ are the posterior mean and covariance of class row k , and M stacks the class means. The first term rewards probability assigned to the correct label. The variance term inside log-sum-exp accounts for uncertainty in each class logit. The KL term penalizes a posterior that moves too far from the prior. For a diagonal Gaussian and unit prior variance, the analytic KL is one half the sum over parameters of $[\sigma^2 + \mu^2 - 1 - \log(\sigma^2)]$. The code minimizes the negative minibatch estimate of this bound.

The training loss does not sample weights: the head moments make the variational bound a single deterministic calculation. At prediction time, this project draws 500 weight matrices $W^{(s)}$ from $q(W)$, computes a softmax probability vector for each, and averages them: $p(y \mid x, D) \approx (1/S) \sum_s \text{softmax}(W^{(s)} \phi(x))$. This Monte Carlo approximation is distinct from the sampling-free training objective.

From prompts to an executable implementation

Codex generated and edited `vbll_digits_classification.py` from a prompt that specified the method, train/validation/test discipline, matched deterministic baseline, probability metrics, fixed seeds, and machine-readable outputs. The ten-seed runner and report were also generated with AI assistance. No source code was supplied or manually authored by the student in this workflow; the human contribution was to provide the assignment brief and requested repository. The script is an independent small reproduction of the discriminative VBLL objective. It does not import the authors' package and does not claim to reproduce the paper's full benchmark suite.

Implementation stage	What the generated code does	Why it matters
Data preparation	Load digits, cast features to float32, divide intensities by 16, and split stratified labels	Same inputs and class proportions are used in both methods
Deterministic representation	Train 64 → 32 ReLU → 10 MLP with cross-entropy and Adam; select checkpoint by validation NLL	Provides learned features and a directly matched baseline
Variational head	Freeze hidden features; augment with bias; initialize posterior means from the trained head	Follows a post-training Bayesian-last-layer design
Objective and optimizer	Optimize the Jensen classification bound plus analytic KL/N with Adam	Implements the paper-derived variational objective without sampling in the loss
Prediction and metrics	Average 500 posterior softmax samples on test; compute common metrics for both models	Evaluates predictive probabilities, not only top-one labels
Artifacts	Write JSON metrics and charts; repeat via the multi-seed runner	Makes settings and results inspectable and reproducible

AI-assisted verification steps

- The model dimensions were traced end to end: 64 input values, 32 hidden features, one appended bias coordinate, and 10 class-specific posterior rows.
- The posterior mean starts from the same softmax head used by the deterministic model, while the Bayesian head's variances remain trainable.
- The same stratified split and feature network are used for the paired comparison within a seed; test metrics are calculated after validation-based checkpoint selection.
- NumPy, PyTorch, Python, and data-split random generators are seeded. The script enables deterministic PyTorch algorithms and limits CPU threads to one.
- The executed script writes its numerical output as JSON. Report tables are generated from that JSON instead of transcribing metric values by hand.

The code exposes named functions for deterministic seeding, diagonal-Gaussian moments and KL, the classification bound, posterior-predictive sampling, metric evaluation, and the full run. These separations make it possible to compare each implementation stage with the mathematical description. The public source code and complete prompts are linked in the final reproducibility section.

Experimental protocol and evaluation rules

Data, partitions, and repeated runs

The scikit-learn digits data contain 1,797 labeled grayscale images, each represented as 64 values from a low-resolution 8×8 image, with pixel intensities in $[0, 16]$ and ten digit classes [3]. Intensities are divided by 16 to scale the inputs to $[0, 1]$. No data are downloaded at run time.

Within each run, stratified sampling first holds out 25% of the examples for testing. The remaining pool is split again, with 20% reserved for validation. For seed 42 this produces 1,077 training examples, 270 validation examples, and 450 test examples. The same sample counts hold for the other listed seeds. Test partitions from different seeds overlap because they are sampled from this same small dataset; the ten runs are repeated holdouts, not ten independent datasets.

The ten seed values were declared before the repeated run: 11, 23, 42, 57, 99, 123, 2024, 2025, 3024, 4096. Each seed controls the stratified split, model initialization, minibatch order, and posterior-predictive Monte Carlo samples. Both methods in each run share the same data split and deterministic feature extractor. The seed-42 results are retained separately as a transparent single-run example; the summary table reports all ten seeds.

Architecture and optimization settings

Component	Specification
Feature model	64 inputs → 32-unit fully connected ReLU → 10-class linear output
Baseline loss and optimizer	Cross-entropy; Adam; learning rate 0.01; weight decay 0.0001
Batching and stopping	Batch size 128; up to 100 epochs; stop after 15 epochs without validation improvement
Checkpoint selection	MLP: validation NLL. VBLL: validation predictive NLL with 64 fixed-seed posterior samples
VBLL posterior	Independent diagonal Gaussian row for each class; prior variance 1.0; posterior variance initialized to 0.1; output-noise variance 0
VBLL post-training	Freeze MLP features; initialize posterior mean from the MLP classifier; optimize the variational head with Adam at 0.01
Final prediction	500 fixed-seed posterior samples for each test run; average softmax probabilities
Environment	Python 3.13.0; PyTorch 2.12.0+cpu; scikit-learn 1.7.2; NumPy 2.1.3; CPU execution

What the reported metrics mean

- Accuracy is the fraction of test images whose highest-probability class matches the label; higher is better.
- Negative log likelihood (NLL) averages the negative log probability assigned to the correct class; lower values penalize confident mistakes strongly.
- The multiclass Brier score averages the sum of squared differences between the probability vector and one-hot target; lower is better.
- Expected calibration error (ECE) uses 15 confidence bins and weights the absolute accuracy-confidence gap in each bin by that bin's share of examples; lower is better. ECE depends on binning and sample size.
- Error-detection AUROC treats a wrong prediction as the positive event and predictive entropy as the ranking score; higher means errors tend to receive higher uncertainty scores.

Results across one fixed split and ten seeds

A detailed look at the seed-42 test set

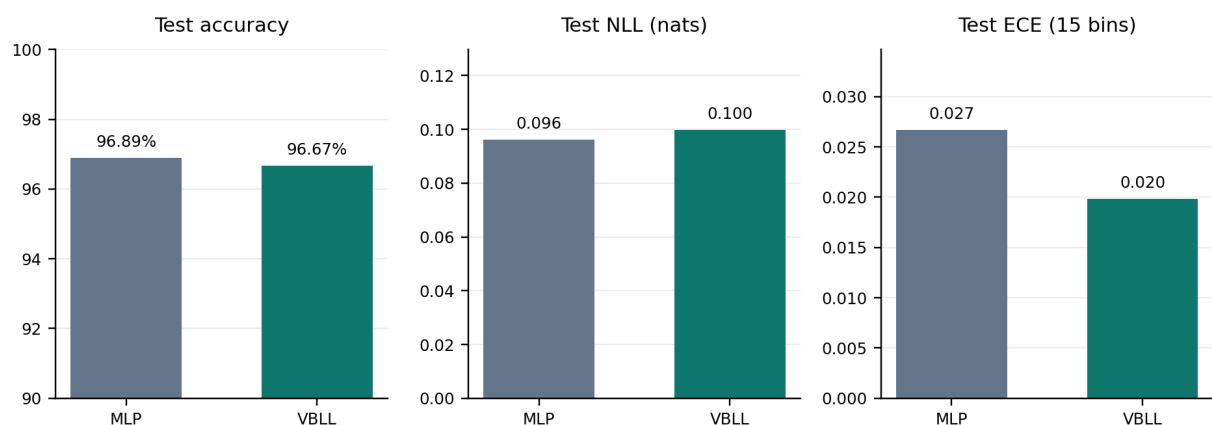
The seed-42 split is useful for tracing one actual execution and is not presented as the sole basis for conclusions. It contains 450 held-out images. The validation split selected the checkpoints; those validation observations did not enter the final test metrics.

Method	Accuracy	NLL	Brier	ECE-15	Error AUROC	Entropy
Deterministic MLP	96.89%	0.0962	0.0475	0.0267	0.9517	0.076
VBLL	96.67%	0.0998	0.0488	0.0198	0.9523	0.145

Seed 42, 450 held-out images. Entropy is in nats; lower is not automatically better because useful uncertainty should track errors.

On this one split, the MLP accuracy was 96.89% and VBLL accuracy was 96.67%. VBLL's ECE was lower (0.0198 versus 0.0267), while NLL and Brier score were slightly higher. The apparent ECE advantage in this split does not persist across the ten-run average, which is exactly why conclusions should not be based on a convenient single random partition.

Digits test set comparison (450 images)



Seed-42 accuracy, NLL, and ECE. This plot describes that split only.

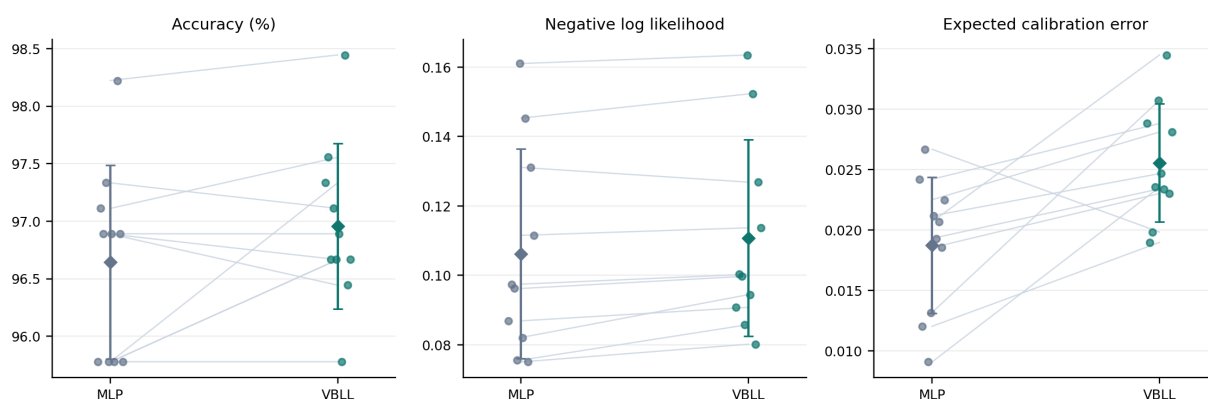
Ten-run summary

For each metric, the table below shows the arithmetic mean and sample standard deviation across the ten seeds. The MLP and VBLL values are paired by seed. These dispersions describe sensitivity to the chosen split and training randomness under this protocol; because splits reuse examples, they are not confidence intervals from independent samples.

Metric	Deterministic MLP mean $\pm$ SD	VBLL mean $\pm$ SD	Direction favored
Accuracy	96.64 $\pm$ 0.84%	96.96 $\pm$ 0.72%	higher
NLL (nats)	0.1062 $\pm$ 0.0302	0.1108 $\pm$ 0.0283	lower
Brier score	0.0490 $\pm$ 0.0101	0.0503 $\pm$ 0.0101	lower
ECE (15 bins)	0.0187 $\pm$ 0.0056	0.0256 $\pm$ 0.0049	lower
Error AUROC	0.9587 $\pm$ 0.0224	0.9512 $\pm$ 0.0185	higher

Ten stratified holdout runs. Accuracy is expressed as percent; all other metrics are unitless except NLL/entropy, which use nats.

Paired results across 10 stratified random splits (mean $\pm$ sample SD)



Paired test results for ten seeds. Each faint segment joins the two methods on the same split; diamonds show mean $\pm$ one sample standard deviation.

Paired differences and consistency

Metric	VBLL – MLP mean $\pm$ SD	VBLL better	Preferred direction
Accuracy	+0.31 $\pm$ 0.63 pp	5/10	higher
NLL (nats)	+0.0045 $\pm$ 0.0046	1/10	lower
Brier score	+0.0014 $\pm$ 0.0022	3/10	lower
ECE (15 bins)	+0.0068 $\pm$ 0.0070	1/10	lower
Error AUROC	-0.0075 $\pm$ 0.0110	2/10	higher

Positive accuracy difference favors VBLL; negative differences favor VBLL for lower-is-better metrics. "VBLL better" counts strict wins among the ten paired runs.

VBLL was more accurate in 5 of 10 runs, with the mean paired gain of +0.31 percentage points small relative to its 0.63-point run-to-run standard deviation. Probability-quality metrics do not support a consistent VBLL advantage here: VBLL had lower NLL in 1/10 runs, lower Brier score in 3/10, lower ECE in 1/10, and higher error-detection AUROC in 2/10.

VBLL's mean predictive entropy was 0.153 $\pm$ 0.016 nats, compared with 0.103 $\pm$ 0.020 for the MLP. Its predictions were less concentrated on average, but greater entropy alone is not proof of better uncertainty estimates. The lower aggregate error-detection AUROC indicates that, in this setup, the extra uncertainty did not consistently rank the misclassified digits more effectively.

Interpretation, limitations, and next steps

The most defensible result is that this small implementation preserves high digit accuracy and learns a nonzero distribution over classifier weights, while its predictive probability quality is mixed against the deterministic MLP. Seed 42 alone made ECE look better for VBLL; the ten-run mean reversed that direction. The repeated runs therefore improved the reliability of the interpretation even though they did not make the Bayesian method appear uniformly superior.

Limit	Effect on interpretation	Useful follow-up
Small, easy, low-resolution dataset	Results may not transfer to natural images or high-dimensional vision tasks	Re-run on a larger image benchmark with a fixed documented preprocessing pipeline

Limit	Effect on interpretation	Useful follow-up
Repeated holdout partitions overlap	Mean $\pm$ SD is descriptive; do not present it as a formal independent-sample confidence interval	Use a prespecified external test set or nested resampling with dependence-aware analysis
Frozen deterministic features	Uncertainty is limited to the last layer; feature uncertainty is omitted	Compare post-training VBLL with joint variational feature learning
Diagonal Gaussian posterior	Posterior correlations between output weights are excluded	Evaluate a richer covariance approximation if resources permit
Finite 500-sample predictive estimate	Monte Carlo error remains in the reported probabilities	Repeat predictions with more samples and report convergence/sampling sensitivity
ECE uses 15 bins	Calibration score can change with bin boundaries and sample size	Add reliability diagrams and a second calibration summary, while retaining NLL/Brier

Lessons from using AI for the full workflow

This assignment made several practical lessons concrete:

- A precise prompt is a specification: naming the Bayesian component, primary-source requirement, baseline, data split, metrics, and no-test-tuning rule produced a more useful plan than asking only for “a Bayesian classifier.”
- AI output needs external checks. The ICLR proceedings page and author code were used to verify the method identity and implementation context, while the objective and tensor dimensions were traced against the report and code.
- Reproducibility is part of the result. Fixed seeds, stratification, validation-only checkpoint selection, explicit package versions, JSON outputs, and a public source link make it possible for another person to inspect the work.
- A Bayesian label does not guarantee better accuracy or calibration. The model expresses posterior uncertainty, but the choice of features, prior, approximate posterior, data regime, and evaluation metric still determine practical performance.
- Repeated splits can overturn an attractive one-run story. The seed-42 ECE comparison and ten-seed ECE average point in opposite directions; reporting both prevents selective interpretation.
- The student remains responsible for understanding and checking AI-generated work. In particular, a submission should follow the course's AI-use and citation rules, and the learner should be able to explain the model, objective, code, and limitations orally.

The final reflection is not that AI removes the need to understand an assignment. It shifts effort toward specifying the task, checking sources and implementation details, reproducing outputs, and judging the evidence without overstating it.

Reproducing the work and finding the code

The public repository includes the report, generated implementation, ten-seed runner, exact AI prompt log, result JSON, and figures. The assignment's requested source-code webpage is the direct GitHub file link below; the repository root contains setup instructions and the full project.

[AI-generated VBLL experiment source code](#)

[Complete project repository](#)

Reproduction commands

```
python -m pip install -r requirements.txt
python vbll_digits_classification.py
python run_multiseed_experiments.py
```

The single-run command regenerates the seed-42 JSON and figure. The multi-seed command reruns the ten predeclared seeds and regenerates experiment_results.json with both the seed-42 detailed result and the ten-run summary, plus multiseed_metrics.png. The run requires no network access after the listed packages are installed because the digits dataset is bundled with scikit-learn.

References and linked resources

- [1] J. Harrison, J. Willes, and J. Snoek. Variational Bayesian Last Layers. International Conference on Learning Representations (ICLR), 2024. [ICLR proceedings paper](#)
- [2] Vector Institute. VBLL: author-maintained implementation and documentation repository. [Author code](#)
- [3] Scikit-learn developers. load_digits dataset documentation. [Dataset reference](#)